

PROLOG PROGRAMMING LANGUAGE

Programming in Logic

A Comprehensive Guide to Logic Programming and AI Applications

Prepared by Dennis Muthui

Computer Science Instructor | DennisCreatives Data Solutions Ltd
Nairobi, Kenya • 2026 Edition

Table of Contents

Table of Contents.....	2
1. Introduction to Prolog.....	4
1.1 What is Prolog?.....	4
1.2 Key Features of Prolog.....	4
1.3 Prolog Implementations.....	5
1.4 Installing and Running SWI-Prolog.....	7
2. Basic Concepts and Syntax.....	9
2.1 Prolog Terms.....	9
2.2 Facts.....	10
2.3 Rules.....	11
2.4 Queries.....	12
3. Unification and Pattern Matching.....	13
3.1 What is Unification?.....	13
3.2 Unification Rules.....	14
3.3 Unification vs Arithmetic Equality.....	14
4. Backtracking and Search.....	16
4.1 How Backtracking Works.....	16
4.2 The Execution Model.....	16
4.3 The Cut Operator (!).....	17
4.4 Negation as Failure.....	18
5. Lists.....	19
5.1 List Basics.....	19
5.2 Essential List Predicates.....	19
5.3 List Processing Patterns.....	20
5.4 Higher-Order List Predicates (SWI-Prolog).....	23
6. Arithmetic and Operators.....	24
6.1 Arithmetic Evaluation.....	24
7. Recursion.....	26
7.1 Recursion in Prolog.....	26
7.2 Tail Recursion Optimization.....	27
8. Input/Output and String Processing.....	29
8.1 Console I/O.....	29
8.2 File I/O.....	29
8.3 Atom and String Processing.....	30

9. Compound Structures and Trees	32
9.1 Compound Terms as Data Structures.....	32
9.2 Binary Trees	32
10. Dynamic Predicates and Database Manipulation	34
10.1 Assert and Retract.....	34
10.2 Practical Database Examples	34
11. Building Expert Systems in Prolog	36
11.1 Expert System Architecture	36
11.2 Animal Identification Expert System.....	36
11.3 Medical Diagnosis Expert System.....	39
12. Constraint Logic Programming	41
12.1 CLP(FD) - Constraints over Finite Domains.....	41
12.2 CLP(FD) Constraint Operators	42
13. Definite Clause Grammars (DCGs)	43
13.1 What are DCGs?	43
13.2 DCGs with Arguments (Semantic Analysis)	44
14. Debugging, Testing, and Best Practices.....	45
14.1 The Prolog Debugger	45
14.2 Unit Testing with PIUnit.....	46
14.3 Best Practices.....	46
14.4 Common Errors and Solutions.....	47

1. Introduction to Prolog

1.1 What is Prolog?

Prolog (Programming in Logic) is a declarative, logic-based programming language that was developed in 1972 by Alain Colmerauer and Philippe Roussel at the University of Aix-Marseille in France. Unlike imperative languages where you tell the computer how to solve a problem step by step, in Prolog you describe what the problem is by defining facts and rules, and the Prolog engine automatically finds solutions through logical inference.

Prolog is one of the two classical artificial intelligence programming languages (the other being LISP). It is rooted in formal logic – specifically first-order predicate logic – and its execution model is based on theorem proving through a process called resolution and unification. Prolog has been extensively used in artificial intelligence, natural language processing, expert systems, databases, symbolic computation, and automated reasoning.

The key paradigm shift in Prolog is that the programmer defines relationships and constraints rather than algorithms. The built-in inference engine uses backtracking search to explore possible solutions, making Prolog particularly powerful for problems involving searching, pattern matching, and symbolic reasoning.

1.2 Key Features of Prolog

- **Declarative Programming:** You describe what is true (facts and rules) rather than how to compute results. The Prolog engine determines the execution strategy.
- **Automatic Backtracking:** When a computation path fails, Prolog automatically backtracks to try alternative solutions without explicit programmer control.
- **Unification:** Prolog's fundamental operation. Two terms are unified by finding a consistent substitution of variables that makes them identical.
- **Pattern Matching:** Powerful built-in pattern matching is used to select applicable clauses and decompose data structures.
- **Recursive Data Structures:** Lists and trees are naturally represented and manipulated through recursive definitions.
- **Symbolic Computation:** Prolog excels at manipulating symbols, structures, and relationships rather than purely numerical computation.

- **Built-in Search:** Depth-first search with backtracking is the default execution strategy, automatically exploring the solution space.
- **Dynamic Knowledge Base:** Facts and rules can be added or removed at runtime, making Prolog ideal for expert systems.
- **Relational:** Knowledge is expressed as relations (predicates), making it natural for database-like queries and reasoning.

1.3 Prolog Implementations

Implementation	License	Key Strengths
SWI-Prolog	BSD/LGPL	Most popular. Rich libraries, web server, IDE, active community. Ideal for learning and production.
GNU Prolog	GPL/LGPL	Native-code compiler, good constraint solving, finite domain solver built-in.
SICStus Prolog	Commercial	High-performance, ISO-compliant, industrial strength. Used in aerospace and telecommunications.
YAP Prolog	Artistic/GPL	High-performance, Prolog-based machine learning (Aleph ILP system).
Tau Prolog	BSD	JavaScript-based Prolog for browsers and Node.js. Great for web applications.
Scriyer Prolog	BSD	Modern, ISO-conformant implementation written in Rust.
XSB Prolog	LGPL	Tabled resolution (avoids infinite loops), HiLog, strong for deductive databases.
ECLiPSe	MPL	Constraint logic programming, optimization, scheduling applications.

Recommended for Learning

SWI-Prolog is the recommended implementation for students and beginners. It is free, cross-platform, has excellent documentation, an integrated editor (PceEmacs), a graphical debugger, and the largest community. Download from <https://www.swi-prolog.org>

1.4 Installing and Running SWI-Prolog

Platform	Installation
Windows	Download the installer from swi-prolog.org . Run the .exe and follow the wizard. Adds swipl to the system PATH.
macOS	Install via Homebrew: <code>brew install swi-prolog</code> . Or download the .dmg from the website.
Linux (Ubuntu/Debian)	<code>sudo apt-get install swi-prolog</code> . Or use the official PPA for the latest version.
Online	SWISH (https://swish.swi-prolog.org) provides a web-based Prolog environment requiring no installation.

The Interactive Top-Level

```
$ swipl
Welcome to SWI-Prolog (threaded, 64 bits, version 9.2.5)

?- write('Hello, Prolog!'), nl.
Hello, Prolog!
true.

?- X = 5, Y is X * 3.
X = 5,
Y = 15.

?- halt.
$
```

Command	Description
?-	The Prolog prompt – where you type queries (goals)
.	Period terminates every clause and query
;	Type ; after a solution to request more solutions (OR / next)
Enter	Press Enter after a solution to accept it and stop searching

[file].	Consult (load) a Prolog source file: ?- [myfile].
consult(file).	Alternate syntax: ?- consult('myfile.pl').
listing.	Display all loaded clauses
listing(pred/N).	Display clauses for predicate pred with arity N
trace.	Enable step-by-step execution tracing (debugger)
notrace.	Disable tracing
halt.	Exit Prolog
help(Topic).	Display help documentation for a topic

2. Basic Concepts and Syntax

2.1 Prolog Terms

Everything in Prolog is a term. Terms are the fundamental building blocks of all Prolog programs and data. Understanding the term hierarchy is essential for writing correct Prolog programs.

Term Type	Description	Examples
Atom	A constant beginning with a lowercase letter, or any characters enclosed in single quotes	hello, x, john, 'Hello World', 'an atom', +, -
Number	Integer or floating-point number	42, -7, 3.14, 0, -2.5
Variable	Begins with an uppercase letter or underscore. Represents an unknown value.	X, Name, _temp, Student, _
Anonymous Variable	The underscore _ is a special variable that matches anything and is never bound	_ (used when the value is not needed)
Compound Term	A functor (atom) applied to arguments: functor(arg1, arg2, ...)	father(john, mary), point(3, 4), +(2, 3)
List	An ordered collection of terms enclosed in brackets	[1, 2, 3], [a, b, c], [Head Tail]
String	A sequence of characters in double quotes (list of character codes or atom depending on flags)	"hello", "Prolog"

```
% Atoms - constants
kenya.           % lowercase atom
'Nairobi'.       % quoted atom (can contain spaces/uppercase)
a.               % single-character atom

% Numbers
42.              % integer
3.14.            % float
```

```
% Variables - start with uppercase or _
X.                % variable
StudentName.      % variable
_Temp.            % variable (conventional: unused)
_..               % anonymous variable

% Compound terms (structures)
student(alice, 21, cs).      % functor: student, arity: 3
point(3, 4).                 % functor: point, arity: 2
date(2026, february, 6).    % functor: date, arity: 3
tree(node, tree(left), tree(right)). % nested compound term
```

Naming Convention

In Prolog, atoms (constants and predicate names) start with lowercase letters, while variables start with uppercase letters or underscores. This convention is not optional – it is a fundamental syntactic rule that the Prolog parser enforces.

2.2 Facts

Facts are the simplest form of Prolog clause. A fact states that a relationship holds between certain objects. Facts are unconditionally true and form the database of known information. Each fact is a predicate (relation name) followed by its arguments in parentheses, terminated with a period.

```
% Facts define relationships that are unconditionally true

% Unary facts (properties)
male(john).
male(bob).
female(mary).
female(alice).

% Binary facts (relationships between two entities)
parent(john, bob).      % john is a parent of bob
parent(john, alice).    % john is a parent of alice
parent(mary, bob).
parent(mary, alice).

% Facts with more arguments
student(alice, 21, 'Computer Science', 3.7).
student(bob, 23, 'Data Analytics', 3.2).
```

```

student(carol, 20, 'Cybersecurity', 3.9).

teaches(muthui, 'Artificial Intelligence').
teaches(muthui, 'Database Systems').

% Location facts
located_in(nairobi, kenya).
located_in(mombasa, kenya).
located_in(kenya, africa).

```

2.3 Rules

Rules define conditional relationships. A rule has a head (the conclusion) and a body (the conditions). The head is true if all the goals in the body are true. Rules use the `:-` operator, which is read as "if". Commas in the body represent logical AND (conjunction), and semicolons represent logical OR (disjunction).

```

% Rule syntax:  Head :- Body.
% Read as:      Head is true IF Body is true.

% Simple rules
father(X, Y) :- male(X), parent(X, Y).
    % X is the father of Y if X is male AND X is a parent of Y

mother(X, Y) :- female(X), parent(X, Y).
    % X is the mother of Y if X is female AND X is a parent of Y

sibling(X, Y) :-
    parent(Z, X),
    parent(Z, Y),
    X \= Y.
    % X and Y are siblings if they share a parent Z
    % and X is not equal to Y

grandparent(X, Z) :-
    parent(X, Y),
    parent(Y, Z).
    % X is grandparent of Z if X is parent of Y
    % and Y is parent of Z

ancestor(X, Y) :- parent(X, Y).           % Base case
ancestor(X, Y) :- parent(X, Z), ancestor(Z, Y). % Recursive
    % X is an ancestor of Y if X is a parent of Y,
    % OR X is a parent of Z and Z is an ancestor of Y

```

```
% Rule with OR (semicolon)
child_or_teen(X) :-
    age(X, Age),
    (Age < 13 ; (Age >= 13, Age < 18)).
```

2.4 Queries

Queries ask Prolog to prove whether a goal is true. When you type a query at the prompt, Prolog searches its database of facts and rules to find a proof. If variables are present in the query, Prolog finds values for those variables that make the query true.

```
% Simple yes/no queries
?- male(john).
true.

?- male(mary).
false.

% Queries with variables - Prolog finds solutions
?- parent(john, X).
X = bob ;           % Type ; for more solutions
X = alice.

% Complex queries
?- father(X, Y).
X = john,
Y = bob ;
X = john,
Y = alice.

% Query with multiple goals
?- parent(X, bob), parent(X, alice).
X = john ;
X = mary.

% Finding all solutions
?- findall(X, parent(john, X), Children).
Children = [bob, alice].

% Counting solutions
?- aggregate_all(count, parent(john, _), Count).
Count = 2.
```

3. Unification and Pattern Matching

3.1 What is Unification?

Unification is the most fundamental operation in Prolog. When Prolog tries to match a query against facts and rules, it uses unification to determine if two terms can be made identical by substituting variables with appropriate values. Unification is what makes Prolog's pattern matching and logical inference possible.

Two terms unify if they are identical, or if they can be made identical by substituting variables. The `=` operator tests whether two terms unify.

```
% Unification examples
?- X = 5.
X = 5.                                % Variable unifies with a value

?- hello = hello.
true.                                  % Identical atoms unify

?- hello = world.
false.                                 % Different atoms don't unify

?- f(X, b) = f(a, Y).
X = a,                                % Compound terms unify
Y = b.                                % by unifying corresponding arguments

?- f(X, X) = f(a, b).
false.                                 % X can't be both 'a' and 'b'

?- f(X, X) = f(a, a).
X = a.                                % X is consistently bound to 'a'

?- date(D, M, 2026) = date(6, february, Y).
D = 6,
M = february,
Y = 2026.                             % Deep structural matching

?- [H|T] = [a, b, c, d].
H = a,                                % Head of list
T = [b, c, d].                        % Tail of list

?- [X, Y | Z] = [1, 2, 3, 4, 5].
X = 1,
```

```
Y = 2,
Z = [3, 4, 5].
```

3.2 Unification Rules

Case	Rule	Example
Two atoms	Unify only if they are identical	hello = hello → true
Two numbers	Unify only if they are the same number	42 = 42 → true
Variable + Term	Variable is bound to the term	X = hello → X = hello
Two variables	Become linked (co-references)	X = Y → X = Y (aliased)
Compound terms	Unify if same functor/arity and all args unify	f(a,X) = f(a,b) → X = b
Lists	Unify element by element (or Head Tail pattern)	[1,2] = [X,Y] → X=1, Y=2
Different types	Fail to unify	42 = hello → false

3.3 Unification vs Arithmetic Equality

```
% = is UNIFICATION (structural matching)
?- X = 3 + 4.
X = 3+4.           % X is bound to the STRUCTURE +(3,4)
                   % NOT the number 7!

% 'is' evaluates arithmetic expressions
?- X is 3 + 4.
X = 7.             % X is bound to the RESULT 7

% := is arithmetic equality (evaluates both sides)
?- 3 + 4 := 2 + 5.
true.              % Both evaluate to 7

% \= is arithmetic inequality
?- 3 + 4 \= 2 + 6.
true.              % 7 \= 8

% \= is NOT unifiable (not the same as arithmetic \=)
?- hello \= world.
true.              % They cannot unify
```

```
% == is structural identity (no binding)
?- X == Y.
false.           % X and Y are different unbound variables
```

Operator	Name	Description
=	Unification	Attempts to unify two terms (may bind variables)
\=	Not Unifiable	Succeeds if two terms cannot be unified
==	Structural Equality	True if terms are identical WITHOUT binding variables
\==	Structural Inequality	True if terms are not structurally identical
is	Arithmetic Evaluation	Evaluates the right-hand side and unifies with left
:=	Arithmetic Equal	True if both sides evaluate to the same number
:=	Arithmetic Not Equal	True if both sides evaluate to different numbers
<	Less Than	Arithmetic comparison
>	Greater Than	Arithmetic comparison
<=	Less or Equal	Note: <= not <=
>=	Greater or Equal	Arithmetic comparison

Common Pitfall

The less-than-or-equal operator in Prolog is <= (not <=). This is one of the most common syntax errors for programmers coming from other languages. Similarly, not-equal is := for arithmetic and \= for unification.

4. Backtracking and Search

4.1 How Backtracking Works

Backtracking is Prolog's fundamental mechanism for finding solutions. When Prolog tries to satisfy a goal, it searches through clauses top-to-bottom. If a path leads to failure, Prolog automatically undoes variable bindings and tries the next alternative. This process continues until a solution is found or all alternatives are exhausted.

```
% Knowledge base
likes(alice, chocolate).
likes(alice, vanilla).
likes(bob, chocolate).
likes(bob, strawberry).

% Query: What does alice like?
?- likes(alice, X).
X = chocolate ;           % First match found
X = vanilla.              % Backtrack: second match found

% Query: Who likes chocolate?
?- likes(Who, chocolate).
Who = alice ;
Who = bob.

% Compound query with backtracking
% Find something both alice and bob like
?- likes(alice, X), likes(bob, X).
X = chocolate.           % Only chocolate is liked by both
```

4.2 The Execution Model

Prolog's execution can be understood as a depth-first search through a tree of possible solutions. Each node in the tree represents a choice point, and each branch represents a different clause that could match the current goal.

1. **Goal Selection:** Prolog selects the leftmost goal in the current query to resolve first.
2. **Clause Selection:** For the selected goal, Prolog tries to match it against clauses in the database, from top to bottom.

3. **Unification:** If a clause head unifies with the goal, variable bindings are made and the clause body becomes the new set of goals.
4. **Success:** If all goals in the query are satisfied, the query succeeds and variable bindings are reported.
5. **Failure and Backtracking:** If a goal cannot be matched, Prolog undoes the most recent variable bindings and tries the next alternative clause (choice point).
6. **Exhaustion:** If all alternatives are exhausted, the entire query fails.

4.3 The Cut Operator (!)

The cut (!) is a special goal that always succeeds but prevents backtracking past the point where it was encountered. Cut is used to control Prolog's search behavior, improve efficiency, and implement committed choices. However, overuse of cut can make programs harder to understand and can break logical correctness.

```
% Without cut: all solutions are explored
grade(Score, 'A') :- Score >= 70.
grade(Score, 'B') :- Score >= 60, Score < 70.
grade(Score, 'C') :- Score >= 50, Score < 60.
grade(Score, 'D') :- Score >= 40, Score < 50.
grade(Score, 'F') :- Score < 40.

% With cut: committed choice (more efficient)
grade(Score, 'A') :- Score >= 70, !.
grade(Score, 'B') :- Score >= 60, !.
grade(Score, 'C') :- Score >= 50, !.
grade(Score, 'D') :- Score >= 40, !.
grade(_, 'F').

% Green cut: doesn't change semantics, only efficiency
max(X, Y, X) :- X >= Y, !.
max(_, Y, Y).

% Red cut: changes the logical meaning (use carefully)
not_member(_, []) :- !.
not_member(X, [_|_]) :- !, fail.
not_member(X, [_|T]) :- not_member(X, T).
```

Cut Type	Description	Recommendation
----------	-------------	----------------

Green Cut	Does not change the set of solutions; only prevents redundant computation	Safe to use; improves efficiency without side effects
Red Cut	Changes the meaning of the program; removing it would give different results	Use cautiously; document why it is needed
No Cut	Relying on mutual exclusivity of conditions	Preferred when conditions are naturally exclusive

4.4 Negation as Failure

Prolog uses "negation as failure" (NAF) to express negation. The built-in predicate `\+` (or `not`) succeeds if its argument goal fails, and fails if its argument succeeds. This is not classical logical negation but a practical approximation based on the closed-world assumption.

```
% \+ is 'not provable' (negation as failure)
single(X) :- \+ married(X).
    % X is single if X is NOT provably married

married(john).
married(mary).

?- single(john).
false.           % john is married

?- single(bob).
true.            % bob is not in the married database

% Important: variables must be bound before negation!
% WRONG (will always fail):
bad_example :- \+ parent(X, _), write(X).

% CORRECT (bind X first, then negate):
childless(X) :- person(X), \+ parent(X, _).
```

Critical Warning

Never use negation (`\+`) with unbound variables. Prolog's negation works by trying to prove the goal and reporting the opposite result. If variables are unbound, the behavior is unpredictable and logically unsound. Always ensure variables are bound before passing them to `\+`.

5. Lists

5.1 List Basics

Lists are one of the most important data structures in Prolog. A list is an ordered sequence of elements enclosed in square brackets and separated by commas. Lists can contain any Prolog terms, including other lists (nested lists). Lists are defined recursively as either the empty list `[]` or a head element followed by a tail (which is itself a list).

```
% List examples
[1, 2, 3, 4, 5]           % List of integers
[alice, bob, carol]       % List of atoms
[1, hello, [a, b], 3.14]  % Mixed types (heterogeneous)
[]                         % Empty list
[[1,2], [3,4], [5,6]]    % List of lists (nested)

% Head|Tail notation - the fundamental list decomposition
?- [H | T] = [a, b, c, d].
H = a,
T = [b, c, d].

?- [X, Y | Rest] = [1, 2, 3, 4, 5].
X = 1,
Y = 2,
Rest = [3, 4, 5].

?- [A, B, C] = [one, two, three].
A = one,
B = two,
C = three.

% The dot notation (internal representation)
% [a, b, c] is actually .(a, .(b, .(c, [])))
?- [a|[b|[c|[]]]] = [a, b, c].
true.
```

5.2 Essential List Predicates

```
% ---- MEMBER: Check if element is in list ----
member(X, [X|_]).          % Base: X is the head
member(X, [_|T]) :- member(X, T). % Recursive: X is in tail
```

```

?- member(b, [a, b, c]).
true.
?- member(X, [a, b, c]).
X = a ; X = b ; X = c.

% ---- LENGTH: Count elements ----
my_length([], 0).
my_length([_|T], N) :- my_length(T, N1), N is N1 + 1.

?- my_length([a, b, c], N).
N = 3.

% ---- APPEND: Concatenate two lists ----
my_append([], L, L).
my_append([H|T1], L2, [H|T3]) :- my_append(T1, L2, T3).

?- my_append([1,2], [3,4], Result).
Result = [1, 2, 3, 4].

% Append is REVERSIBLE (a key Prolog feature):
?- my_append(X, Y, [a, b, c]).
X = [], Y = [a,b,c] ;
X = [a], Y = [b,c] ;
X = [a,b], Y = [c] ;
X = [a,b,c], Y = [].

% ---- REVERSE: Reverse a list ----
my_reverse([], []).
my_reverse([H|T], R) :-
    my_reverse(T, RT),
    my_append(RT, [H], R).

% ---- NTH ELEMENT ----
nth_element(1, [H|_], H).
nth_element(N, [_|T], E) :-
    N > 1, N1 is N - 1,
    nth_element(N1, T, E).

% ---- LAST element ----
my_last([X], X).
my_last([_|T], X) :- my_last(T, X).

```

5.3 List Processing Patterns

```

% ---- SUM of a numeric list ----
sum_list([], 0).
sum_list([H|T], Sum) :-

```

```

    sum_list(T, RestSum),
    Sum is H + RestSum.

?- sum_list([10, 20, 30], S).
S = 60.

% ---- MAX element ----
max_list([X], X).
max_list([H|T], Max) :-
    max_list(T, TMax),
    (H > TMax -> Max = H ; Max = TMax).

% ---- FILTER: keep elements matching condition ----
filter_positive([], []).
filter_positive([H|T], [H|R]) :-
    H > 0, !,
    filter_positive(T, R).
filter_positive([_|T], R) :-
    filter_positive(T, R).

?- filter_positive([3, -1, 5, -2, 7], R).
R = [3, 5, 7].

% ---- MAP: apply function to each element ----
double_list([], []).
double_list([H|T], [D|DT]) :-
    D is H * 2,
    double_list(T, DT).

?- double_list([1, 2, 3], R).
R = [2, 4, 6].

% ---- SORT and REMOVE DUPLICATES ----
?- msort([3,1,4,1,5], S).    % msort keeps duplicates
S = [1, 1, 3, 4, 5].
?- sort([3,1,4,1,5], S).    % sort removes duplicates
S = [1, 3, 4, 5].

% ---- FLATTEN nested lists ----
my_flatten([], []).
my_flatten([H|T], Flat) :-
    is_list(H), !,
    my_flatten(H, FH),
    my_flatten(T, FT),
    append(FH, FT, Flat).
my_flatten([H|T], [H|FT]) :-
    my_flatten(T, FT).

```

```
?- my_flatten([1,[2,[3]],4,[5,6]], R).  
R = [1, 2, 3, 4, 5, 6].
```

5.4 Higher-Order List Predicates (SWI-Prolog)

```
% SWI-Prolog provides built-in higher-order predicates

% maplist: apply a predicate to each element
?- maplist(succ, [1,2,3], R).      % succ(X, Y) :- Y is X+1
R = [2, 3, 4].

?- maplist(write, [a, b, c]).
abc
true.

% include/exclude: filter elements
positive(X) :- X > 0.
?- include(positive, [3,-1,5,-2], R).
R = [3, 5].
?- exclude(positive, [3,-1,5,-2], R).
R = [-1, -2].

% foldl: left fold (reduce)
add(X, Acc, Result) :- Result is Acc + X.
?- foldl(add, [1,2,3,4], 0, Sum).
Sum = 10.

% findall, bagof, setof - collect all solutions
?- findall(X, member(X, [a,b,c,a,b]), All).
All = [a, b, c, a, b].

?- setof(X, member(X, [c,a,b,a,c]), Set).
Set = [a, b, c].      % Sorted, no duplicates

% aggregate: compute aggregates
?- aggregate_all(max(Score),
    student(_, _, _, Score), MaxScore).
MaxScore = 3.9.
```

6. Arithmetic and Operators

6.1 Arithmetic Evaluation

Arithmetic in Prolog requires the `is/2` operator to evaluate expressions. Without `is`, arithmetic expressions are treated as unevaluated structures (compound terms). This is because Prolog is fundamentally a symbolic language, not a numeric one.

```
% The 'is' operator evaluates arithmetic expressions
?- X is 3 + 4 * 2.
X = 11.

?- X is (3 + 4) * 2.
X = 14.

?- X is 2 ** 10.           % Exponentiation
X = 1024.

?- X is 17 mod 5.
X = 2.

?- X is 17 // 5.          % Integer division
X = 3.

?- X is abs(-42).
X = 42.

?- X is max(10, 20).
X = 20.

?- X is sqrt(144).
X = 12.0.

?- X is ceiling(3.2).
X = 4.

?- X is truncate(3.9).
X = 3.
```

Operator / Function	Description	Example
<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code>	Basic arithmetic	<code>X is 3 + 4 → X = 7</code>

//	Integer division	X is 7 // 2 → X = 3
mod	Modulo (remainder)	X is 7 mod 2 → X = 1
**	Exponentiation (power)	X is 2 ** 8 → X = 256
abs/1	Absolute value	X is abs(-5) → X = 5
max/2, min/2	Maximum / Minimum	X is max(3, 7) → X = 7
sqrt/1	Square root	X is sqrt(25) → X = 5.0
sin/1, cos/1, tan/1	Trigonometric functions	X is sin(3.14159) → ~0.0
log/1	Natural logarithm	X is log(2.71828) → ~1.0
ceiling/1, floor/1	Rounding	X is ceiling(3.2) → X = 4
round/1	Round to nearest integer	X is round(3.5) → X = 4
truncate/1	Truncate to integer	X is truncate(3.9) → X = 3
float_integer_part/1	Integer part of float	X is float_integer_part(3.7) → 3.0
msb/1	Most significant bit	X is msb(255) → X = 7
∨	Bitwise OR	X is 5 ∨ 3 → X = 7
∧	Bitwise AND	X is 5 ∧ 3 → X = 1
xor	Bitwise XOR	X is 5 xor 3 → X = 6
<<, >>	Bit shift left/right	X is 1 << 3 → X = 8

7. Recursion

7.1 Recursion in Prolog

Recursion is the primary looping mechanism in Prolog. Since Prolog has no traditional for/while loops, all repetitive computations are expressed through recursive predicate definitions. Every recursive predicate must have at least one base case (termination condition) and one recursive case.

```
% ---- FACTORIAL ----
factorial(0, 1).                                % Base case
factorial(N, F) :-
    N > 0,
    N1 is N - 1,
    factorial(N1, F1),
    F is N * F1.

?- factorial(5, F).
F = 120.

% ---- FIBONACCI ----
fibonacci(0, 0).
fibonacci(1, 1).
fibonacci(N, F) :-
    N > 1,
    N1 is N - 1,
    N2 is N - 2,
    fibonacci(N1, F1),
    fibonacci(N2, F2),
    F is F1 + F2.

?- fibonacci(10, F).
F = 55.

% ---- POWER (X^N) ----
power(_, 0, 1).
power(X, N, Result) :-
    N > 0,
    N1 is N - 1,
    power(X, N1, R1),
    Result is X * R1.

% ---- GCD (Euclidean Algorithm) ----
gcd(X, 0, X) :- X > 0.
```

```

gcd(X, Y, G) :-
    Y > 0,
    R is X mod Y,
    gcd(Y, R, G).

?- gcd(12, 8, G).
G = 4.

% ---- COUNTING DOWN ----
countdown(0) :- write('Go!'), nl.
countdown(N) :-
    N > 0,
    write(N), write('... '),
    N1 is N - 1,
    countdown(N1).

?- countdown(3).
3... 2... 1... Go!

```

7.2 Tail Recursion Optimization

Tail recursion occurs when the recursive call is the last operation in a clause. Prolog systems can optimize tail-recursive predicates to use constant stack space (like a loop), preventing stack overflow on large inputs. Always prefer tail-recursive definitions when possible.

```

% NON-tail-recursive factorial (uses O(n) stack)
fact(0, 1).
fact(N, F) :-
    N > 0, N1 is N - 1,
    fact(N1, F1),
    F is N * F1.           % Multiplication AFTER recursive call

% TAIL-recursive factorial with accumulator (O(1) stack)
fact_tail(N, F) :- fact_acc(N, 1, F).

fact_acc(0, Acc, Acc).    % Base: result = accumulator
fact_acc(N, Acc, F) :-
    N > 0,
    NewAcc is Acc * N,
    N1 is N - 1,
    fact_acc(N1, NewAcc, F). % Recursive call is LAST

% Tail-recursive list length
length_tail(List, Len) :- length_acc(List, 0, Len).
length_acc([], Acc, Acc).

```

```
length_acc([_|T], Acc, Len) :-  
    Acc1 is Acc + 1,  
    length_acc(T, Acc1, Len).  
  
% Tail-recursive list reversal (much faster than naive)  
reverse_tail(List, Rev) :- rev_acc(List, [], Rev).  
rev_acc([], Acc, Acc).  
rev_acc([H|T], Acc, Rev) :-  
    rev_acc(T, [H|Acc], Rev).
```

Performance Tip

Always use tail recursion with accumulators for predicates that process large lists or compute values iteratively. The naive (non-tail) recursive version of `reverse/2` is $O(n^2)$ while the accumulator version is $O(n)$.

8. Input/Output and String Processing

8.1 Console I/O

```
% ---- OUTPUT ----
?- write('Hello, World!'), nl.
Hello, World!

?- writeln('Hello, World!'). % write + newline (SWI-Prolog)
Hello, World!

?- format('Name: ~w, Age: ~d~n', ['Alice', 21]).
Name: Alice, Age: 21

% Format specifiers:
% ~w write (any term)      ~d decimal integer
% ~f float                ~e scientific notation
% ~a atom                 ~s string (char codes)
% ~n newline              ~t tab/fill
% ~i ignore argument      ~`Cc fill with character C

?- format('~w scored ~2f out of ~d~n', ['Bob', 87.5, 100]).
Bob scored 87.50 out of 100

% ---- INPUT ----
?- read(X). % Reads a Prolog term (must end with .)
|: hello.
X = hello.

?- read_term(X, []). % More control over reading

% Read a line as a string
?- read_line_to_string(user_input, Line).
|: Hello World
Line = "Hello World".

% Interactive example
ask_name :-
    write('Enter your name: '),
    read_line_to_string(user_input, Name),
    format('Hello, ~w!~n', [Name]).
```

8.2 File I/O

```

% ---- WRITING TO FILE ----
write_to_file(Filename, Data) :-
    open(Filename, write, Stream),
    write(Stream, Data),
    nl(Stream),
    close(Stream).

% Using tell/told (stream redirection)
save_report :-
    tell('report.txt'),
    writeln('=== Student Report ==='),
    writeln('Generated by Prolog'),
    told.                % Close and restore stdout

% ---- READING FROM FILE ----
read_file(Filename) :-
    open(Filename, read, Stream),
    read_lines(Stream),
    close(Stream).

read_lines(Stream) :-
    read_line_to_string(Stream, Line),
    (Line \= end_of_file ->
        writeln(Line),
        read_lines(Stream)
    ;   true).

% Read Prolog terms from file
read_terms(Filename, Terms) :-
    open(Filename, read, S),
    read_all_terms(S, Terms),
    close(S).

read_all_terms(S, []) :-
    at_end_of_stream(S), !.
read_all_terms(S, [T|Ts]) :-
    read(S, T),
    read_all_terms(S, Ts).

```

8.3 Atom and String Processing

```

% ---- ATOM MANIPULATION ----
?- atom_length(hello, N).
N = 5.

?- atom_concat(hello, ' world', R).
R = 'hello world'.

```

```

?- atom_chars(hello, Chars).
Chars = [h, e, l, l, o].

?- atom_string(hello, S).
S = "hello".

?- upcase_atom(hello, U).
U = 'HELLO'.

?- downcase_atom('HELLO', D).
D = hello.

?- sub_atom(abcdef, 2, 3, _, Sub).
Sub = cde.

% ---- STRING OPERATIONS (SWI-Prolog) ----
?- string_concat("Hello", " World", R).
R = "Hello World".

?- string_length("Hello", N).
N = 5.

?- split_string("a,b,c,d", ",", "", Parts).
Parts = ["a", "b", "c", "d"].

?- number_codes(42, Codes).
Codes = [52, 50].

?- number_chars(42, Chars).
Chars = ['4', '2'].

?- term_to_atom(f(a,b), A).
A = 'f(a,b)'.

```

9. Compound Structures and Trees

9.1 Compound Terms as Data Structures

Prolog's compound terms (functors with arguments) serve as the primary data structuring mechanism. They are analogous to records, structs, or objects in other languages but are more flexible due to pattern matching and unification.

```
% Records / Structs
student(name('Alice Wanjiku'), age(21), course(cs), gpa(3.7)).
point(3, 4).
date(2026, february, 6).
address(street('Thika Road'), city(nairobi), country(kenya)).

% Accessing nested structures via unification
get_name(student(name(N), _, _, _), N).
get_gpa(student(_, _, _, gpa(G)), G).

?- S = student(name('Alice'), age(21), course(cs), gpa(3.7)),
   get_name(S, Name),
   get_gpa(S, GPA).
Name = 'Alice',
GPA = 3.7.

% Key-Value pairs using the - operator
?- pairs_keys_values(Pairs, [a,b,c], [1,2,3]).
Pairs = [a-1, b-2, c-3].
```

9.2 Binary Trees

```
% Binary tree representation:
%   nil           - empty tree
%   t(Left, Value, Right) - node with left/right subtrees

% Example tree:
%           5
%        /  \
%       3    8
%      / \   \
%     1  4   9

example_tree(
```

```

    t(t(t(nil,1,nil), 3, t(nil,4,nil)),
      5,
      t(nil, 8, t(nil,9,nil))).

% ---- TREE MEMBERSHIP ----
tree_member(X, t(_, X, _)).
tree_member(X, t(L, _, _)) :- tree_member(X, L).
tree_member(X, t(_, _, R)) :- tree_member(X, R).

% ---- IN-ORDER TRAVERSAL ----
inorder(nil, []).
inorder(t(L, V, R), Result) :-
    inorder(L, LL),
    inorder(R, RL),
    append(LL, [V|RL], Result).

?- example_tree(T), inorder(T, List).
List = [1, 3, 4, 5, 8, 9].

% ---- TREE HEIGHT ----
tree_height(nil, 0).
tree_height(t(L, _, R), H) :-
    tree_height(L, LH),
    tree_height(R, RH),
    H is max(LH, RH) + 1.

% ---- BST INSERT ----
bst_insert(nil, X, t(nil, X, nil)).
bst_insert(t(L, V, R), X, t(NL, V, R)) :-
    X < V, !, bst_insert(L, X, NL).
bst_insert(t(L, V, R), X, t(L, V, NR)) :-
    X > V, !, bst_insert(R, X, NR).
bst_insert(t(L, X, R), X, t(L, X, R)). % Already exists

```

10. Dynamic Predicates and Database Manipulation

10.1 Assert and Retract

Prolog's dynamic database allows facts and rules to be added or removed at runtime. This is essential for building systems that learn, maintain state, or accumulate knowledge during execution. Predicates that will be modified dynamically must be declared as dynamic.

```
% Declare dynamic predicates
:- dynamic student/3.
:- dynamic counter/1.
:- dynamic known/3.

% ---- ASSERT: Add facts at runtime ----
?- assert(student(alice, cs, 3.7)).
true.
?- assert(student(bob, da, 3.2)).
true.

% asserta: add at the BEGINNING of the database
?- asserta(student(carol, cy, 3.9)).

% assertz: add at the END of the database (same as assert)
?- assertz(student(david, se, 3.4)).

% ---- RETRACT: Remove facts at runtime ----
?- retract(student(bob, da, 3.2)).
true.

% Retract with pattern matching
?- retract(student(_, cs, _)). % Remove first CS student

% retractall: Remove ALL matching facts
?- retractall(student(_, _, _)). % Remove all students

% ---- ABOLISH: Remove all clauses of a predicate ----
?- abolish(student/3).
```

10.2 Practical Database Examples

```
% ---- GLOBAL COUNTER ----
:- dynamic counter/1.
```

```

counter(0).

increment_counter :-
    retract(counter(N)),
    N1 is N + 1,
    assert(counter(N1)).

get_counter(N) :- counter(N).

% ---- STUDENT DATABASE with CRUD ----
:- dynamic student/4. % name, age, course, gpa

% Create
add_student(Name, Age, Course, GPA) :-
    (student(Name, _, _, _) ->
        format('Student ~w already exists.~n', [Name]))
    ; assert(student(Name, Age, Course, GPA)),
        format('Added: ~w~n', [Name])
    ).

% Read
show_students :-
    writeln('=== Student List ==='),
    forall(
        student(Name, Age, Course, GPA),
        format(' ~w | Age: ~d | ~w | GPA: ~2f~n',
            [Name, Age, Course, GPA])
    ).

% Update
update_gpa(Name, NewGPA) :-
    retract(student(Name, Age, Course, _)),
    assert(student(Name, Age, Course, NewGPA)),
    format('Updated ~w GPA to ~2f~n', [Name, NewGPA]).

% Delete
remove_student(Name) :-
    retract(student(Name, _, _, _)),
    format('Removed: ~w~n', [Name]).

```

11. Building Expert Systems in Prolog

11.1 Expert System Architecture

Prolog is naturally suited for building expert systems due to its declarative rules, automatic backtracking, and dynamic database capabilities. An expert system in Prolog typically consists of a knowledge base (facts and rules), an inference engine (Prolog itself), a user interface (I/O predicates), and an explanation facility.

11.2 Animal Identification Expert System

```
%%% =====
%%% Animal Identification Expert System
%%% =====

:- dynamic known/3.

% ---- MAIN ENTRY POINT ----
start :-
    writeln('=== Animal Identification System ==='),
    writeln('Answer questions with yes or no. '), nl,
    (identify(Animal) ->
        format('~nThe animal is: ~w~n', [Animal])
    ;   writeln('Could not identify the animal.'))
    ),
    retractall(known(_, _, _)).

% ---- IDENTIFICATION RULES ----
identify(cheetah) :-
    is_mammal, is_carnivore,
    verify(has, tawny_color),
    verify(has, dark_spots).

identify(tiger) :-
    is_mammal, is_carnivore,
    verify(has, tawny_color),
    verify(has, black_stripes).

identify(giraffe) :-
    is_mammal, is_ungulate,
    verify(has, long_neck),
    verify(has, dark_spots).
```

```

identify(zebra) :-
    is_mammal, is_ungulate,
    verify(has, black_stripes).

identify(ostrich) :-
    is_bird,
    verify(does_not, fly),
    verify(has, long_neck).

identify(penguin) :-
    is_bird,
    verify(does_not, fly),
    verify(can, swim),
    verify(has, black_and_white).

identify(eagle) :-
    is_bird,
    verify(can, fly),
    verify(is, carnivorous).

```

```

% ---- CLASSIFICATION HELPERS ----
is_mammal :- verify(has, hair), !.
is_mammal :- verify(does, give_milk).

is_bird :- verify(has, feathers), !.
is_bird :- verify(can, fly), verify(does, lay_eggs).

is_carnivore :- verify(does, eat_meat), !.
is_carnivore :- verify(has, pointed_teeth),
                verify(has, claws).

is_ungulate :- is_mammal, verify(has, hooves), !.
is_ungulate :- is_mammal, verify(does, chew_cud).

% ---- USER INTERACTION ----
ask(Attr, Value) :-
    format('Does the animal have/is/can/does ~w ~w? ',
           [Attr, Value]),
    read_line_to_string(user_input, Reply),
    (sub_string(Reply, 0, _, _, "y") ->
        assert(known(yes, Attr, Value))
    ;   assert(known(no, Attr, Value)), fail
    ).

verify(Attr, Value) :-
    known(yes, Attr, Value), !.
verify(Attr, Value) :-

```

```
\+ known(no, Attr, Value),  
ask(Attr, Value).
```

How It Works

The `verify/2` predicate first checks if the answer is already known (cached in the dynamic database). If not, it asks the user. User responses are stored as `known/3` facts so the same question is never asked twice. The system uses backtracking to try different animal hypotheses until one matches all conditions.

11.3 Medical Diagnosis Expert System

```

%%% =====
%%% Simple Medical Diagnosis System
%%% For educational purposes only
%%% =====

:- dynamic symptom/1.

diagnose :-
    writeln('=== Medical Symptom Checker ==='),
    writeln('Enter symptoms one at a time. '),
    writeln('Type "done" when finished. '), nl,
    collect_symptoms,
    nl, writeln('--- Possible Conditions ---'),
    (diagnose_condition(Condition, Confidence) ->
        format('Condition: ~w (~w confidence)~n',
            [Condition, Confidence]),
        fail % Continue to find more
    ; true),
    retractall(symptom(_)).

collect_symptoms :-
    write('Symptom: '),
    read_line_to_string(user_input, S),
    (S \= "done" ->
        term_to_atom(Term, S),
        assert(symptom(Term)),
        collect_symptoms
    ; true).

diagnose_condition('Common Cold', high) :-
    symptom(runny_nose), symptom(sneezing),
    symptom(sore_throat), \+ symptom(high_fever).

diagnose_condition('Influenza', high) :-
    symptom(high_fever), symptom(body_aches),
    symptom(fatigue).

diagnose_condition('Malaria', high) :-
    symptom(high_fever), symptom(chills),
    symptom(sweating), symptom(headache).

diagnose_condition('Allergies', medium) :-
    symptom(sneezing), symptom(watery_eyes),
    symptom(itching).

diagnose_condition('Migraine', medium) :-
    symptom(severe_headache),

```

```
(symptom(nausea) ; symptom(light_sensitivity)).
```

12. Constraint Logic Programming

12.1 CLP(FD) - Constraints over Finite Domains

Constraint Logic Programming over Finite Domains (CLP(FD)) extends Prolog with the ability to reason about integer variables subject to constraints. Instead of eagerly assigning values, CLP(FD) propagates constraints to narrow down possible values, making it extremely efficient for combinatorial problems like scheduling, puzzles, and resource allocation.

```
:- use_module(library(clpfd)).

% ---- Basic constraint examples ----
?- X in 1..10, X #> 5.
X in 6..10.

?- X in 1..10, Y in 1..10, X + Y #= 15, X #< Y.
X in 5..7, Y in 8..10.

% ---- SEND + MORE = MONEY puzzle ----
send_more_money([S,E,N,D,M,O,R,Y]) :-
    Vars = [S,E,N,D,M,O,R,Y],
    Vars ins 0..9,
    all_different(Vars),
    S #\= 0, M #\= 0,
    1000*S + 100*E + 10*N + D
    + 1000*M + 100*O + 10*R + E
    #= 10000*M + 1000*O + 100*N + 10*E + Y,
    label(Vars).

?- send_more_money(Solution).
Solution = [9,5,6,7,1,0,8,2].

% ---- N-QUEENS PROBLEM ----
n_queens(N, Queens) :-
    length(Queens, N),
    Queens ins 1..N,
    safe_queens(Queens),
    label(Queens).

safe_queens([]).
safe_queens([Q|Qs]) :-
    no_attack(Q, Qs, 1),
    safe_queens(Qs).
```

```

no_attack(_, [], _).
no_attack(Q, [Q1|Qs], D) :-
    Q #\= Q1,
    Q #\= Q1 + D,
    Q #\= Q1 - D,
    D1 is D + 1,
    no_attack(Q, Qs, D1).

?- n_queens(8, Q).
Q = [1, 5, 8, 6, 3, 7, 2, 4].

```

12.2 CLP(FD) Constraint Operators

Operator	Description	Example
#=	Equals	X #= Y + 3
#\=	Not equals	X #\= Y
#<	Less than	X #< 10
#>	Greater than	X #> 0
#=<	Less or equal	X #=< 100
#>=	Greater or equal	X #>= 1
in	Domain membership	X in 1..9
ins	Domain for list	Vars ins 0..9
all_different/1	All variables different	all_different([X,Y,Z])
all_distinct/1	Stronger propagation version	all_distinct([X,Y,Z])
label/1	Enumerate solutions	label(Vars)
labeling/2	Label with strategy	labeling([min(Cost)], Vars)
sum/3	Sum of list with relation	sum(Vars, #=, Total)
scalar_product/4	Weighted sum	scalar_product(Ws, Vs, #=, T)

13. Definite Clause Grammars (DCGs)

13.1 What are DCGs?

Definite Clause Grammars (DCGs) are a notation in Prolog for defining grammars and parsing languages. DCGs use the `-->` operator and are automatically translated into regular Prolog clauses with difference lists. They are extensively used in natural language processing, compiler construction, and protocol parsing.

```
% ---- SIMPLE ENGLISH SENTENCE GRAMMAR ----
sentence --> noun_phrase, verb_phrase.

noun_phrase --> determiner, noun.
noun_phrase --> noun.

verb_phrase --> verb.
verb_phrase --> verb, noun_phrase.

determiner --> [the].
determiner --> [a].
determiner --> [every].

noun --> [student].
noun --> [teacher].
noun --> [course].
noun --> [book].

verb --> [teaches].
verb --> [studies].
verb --> [reads].
verb --> [likes].

% ---- PARSING ----
?- phrase(sentence, [the, student, reads, a, book]).
true.

?- phrase(sentence, [a, teacher, teaches, the, course]).
true.

?- phrase(sentence, [reads, the]).
false.      % Not a valid sentence

% ---- GENERATION ----
```

```
?- phrase(sentence, S).
S = [the, student, teaches] ;
S = [the, student, teaches, the, student] ;
S = [the, student, studies] ;
... % Generates all valid sentences
```

13.2 DCGs with Arguments (Semantic Analysis)

```
% DCG rules can carry arguments for building parse trees
sentence(s(NP, VP)) --> np(NP), vp(VP).
np(np(D, N)) --> det(D), noun(N).
vp(vp(V)) --> verb(V).
vp(vp(V, NP)) --> verb(V), np(NP).
det(det(the)) --> [the].
det(det(a)) --> [a].
noun(n(student)) --> [student].
noun(n(book)) --> [book].
verb(v(reads)) --> [reads].

?- phrase(sentence(Tree),
           [the, student, reads, a, book]).
Tree = s(np(det(the), n(student)),
         vp(v(reads), np(det(a), n(book)))).
```

14. Debugging, Testing, and Best Practices

14.1 The Prolog Debugger

SWI-Prolog includes a powerful four-port box model debugger (based on the Byrd box model). It traces execution through four ports: Call (entering a goal), Exit (goal succeeded), Redo (backtracking into a goal), and Fail (goal failed).

Command	Description
trace.	Enable the debugger (step through all goals)
notrace.	Disable the debugger
spy(pred/N).	Set a spy point on a predicate (breakpoint)
nospy(pred/N).	Remove a spy point
nospyall.	Remove all spy points
debug.	Enable debug mode (stops at spy points only)
nodebug.	Disable debug mode
gtrace.	Start graphical debugger (SWI-Prolog)
listing(pred/N).	Display the clauses of a predicate

Debugger Commands During Trace

Key	Action
c / Enter	Creep - step to next port
s	Skip - run without showing internal steps
l	Leap - run to next spy point
r	Retry - restart the current goal
f	Fail - force the current goal to fail
a	Abort - stop execution
w	Write - display the current goal
p	Print - print with depth limit

g	Goals - show the ancestor goal stack
n	Nodebug - turn off debugging

14.2 Unit Testing with PIUnit

```
:- use_module(library(plunit)).

:- begin_tests(my_predicates).

test(factorial_0) :- factorial(0, 1).
test(factorial_5) :- factorial(5, 120).
test(factorial_10) :- factorial(10, 3628800).

test(member_found) :- member(b, [a, b, c]).
test(member_not_found, [fail]) :- member(z, [a, b, c]).

test(append_basic) :-
    append([1,2], [3,4], [1,2,3,4]).

test(reverse_list) :-
    reverse([1,2,3], [3,2,1]).

test(length_empty) :- length([], 0).
test(length_three) :- length([a,b,c], 3).

:- end_tests(my_predicates).

% Run tests:
?- run_tests.
% test my_predicates:factorial_0 ... passed
% test my_predicates:factorial_5 ... passed
% ... All 9 tests passed
```

14.3 Best Practices

- **Use Declarative Style:** Write predicates that describe relationships, not procedures. Let Prolog's inference engine handle the control flow.
- **Prefer Tail Recursion:** Use accumulators to make recursive predicates tail-recursive for better performance and to avoid stack overflow.
- **Avoid Side Effects:** Minimize the use of assert/retract, I/O, and cut. Pure logical predicates are easier to test and reason about.

- **Use Meaningful Names:** Name predicates as relations (parent_of, is_member, has_property) rather than actions.
- **Document Predicates:** Use comments to document the predicate's purpose, arguments, and modes (+input, -output, ?either).
- **Test with Multiple Modes:** Prolog predicates can often run in multiple directions. Test append/3 both for concatenation and for splitting.
- **Put Base Cases First:** Define base cases before recursive cases to ensure termination and readability.
- **Avoid Singleton Variables:** Replace unused variables with `_` (anonymous variable) to avoid compiler warnings.
- **Use CLP(FD) for Integers:** When working with integer constraints, use CLP(FD) instead of generate-and-test.
- **Use Libraries:** SWI-Prolog has extensive built-in libraries. Check `library(lists)`, `library(apply)`, `library(aggregate)` before writing custom predicates.

14.4 Common Errors and Solutions

Error / Problem	Likely Cause	Solution
Predicate not found	Not loaded or misspelled	Check filename, use <code>[file].</code> or <code>consult(file)</code> .
Singleton variable warning	Variable used only once	Replace with <code>_</code> or prefix with <code>_Var</code>
Infinite loop	Missing base case or non-terminating recursion	Add base case; ensure recursive argument decreases
Stack overflow	Deep non-tail recursion	Convert to tail recursion with accumulator
Unexpected failure	Unification fails silently	Use <code>trace/spy</code> to identify where matching fails
<code>=<</code> not recognized	Using <code><=</code> instead of <code>=<</code>	Prolog uses <code>=<</code> for less-or-equal, not <code><=</code>
Arithmetic error	Unbound variable in 'is'	Ensure right side of 'is' has no unbound variables
Cannot modify static predicate	Predicate not declared dynamic	Add <code>:- dynamic pred/arity.</code> declaration

Arguments not sufficiently instantiated	Calling arithmetic on unbound vars	Use CLP(FD) or bind variables before arithmetic
---	------------------------------------	---

Further Learning Resources

1. "The Art of Prolog" by Sterling and Shapiro – The classic comprehensive textbook.
2. "Programming in Prolog" by Clocksin and Mellish – Excellent introductory text.
3. "Learn Prolog Now!" (learnprolognow.org) – Free online interactive tutorial.
4. SWI-Prolog documentation (swi-prolog.org/pldoc/) – Complete reference.
5. "The Power of Prolog" by Markus Triska (metalevel.at/prolog) – Modern, CLP-focused.